VG101 Lab 7 - Arcade!

Author: Me again, Zhong Zhiyao. Still, contact me if you have any questions about this lab.

Introduction

Have you ever played an arcade game? An arcade game takes player input from its controls, processes it through electrical or computerized components, and displays output to an electronic monitor. In the 1990s, the pace of reform and opening up brought electronic games into the lives of Chinese people. During that period, you can find many arcade video games housed in an arcade cabinet. Besides the most popular ones (Street Fighter, Warriors of Fate, ...), the Mahjong arcade game is also well-known to many people born in the 1970s and 1980s. However, arcade games ultimately declined as competing home video game consoles increased in their graphics and gameplay capability and decreased in cost.



Mahjong Electromagnetic Base (Dynax, Japan, 1989), one of the oldest Mahjong arcade game

One typical feature of the Arcade Mahjong game is that there are only 2 players instead of 4. Every round, you fight with an arcade AI (usually takes the shape of an anime girl or a fascinating lady) to contend for the first one to win.

In this lab, we adopt the rule of the 2-player Arcade Mahjong Game and build a game platform for you (check `Lab7_Stater_Files.zip`). Your task is to implement an AI for the game in the file `brain.cpp`. Your AI will be evaluated on how **fast** and **big** (the score of the winning hand) it performs. You will learn how to write C++ class, build and use data structures, and have a taste of AI programming.

Rules

- I draw a flow chart diagram to help you better understand how the game works. See Appendix.
- The basic rules (tiles used, scoring, draw & discard) are identical to the simplified version of Sichuan Mahjong introduced in Lab 6.
- Before the start of a round, all 108 tiles are shuffled and stacked to form a wall, then each player will receive 13 tiles as the initial hand.
- When a round begins, two players take turns to draw a tile from the wall to his/her hand, and then discard a tile from his/her hand to the river. There's no Chow, Pong, or Kong.
- A round always ends with the winning of a player (**Win by Self Draw** or **Win by Discard**) or running out of tiles (**Game Draw**).
    - **Win by Self Draw**: complete a winning hand by drawing a tile from the wall in the player's turn.
    - **Win by Discard**: complete a winning hand using the tile discarded by another player.
    - **Game Draw**: run out of empty of tiles before any player could win.

- Your opponent is always an AI that does nothing but throws a relatively random tile every turn, and will never claim to win. So you don't need to worry that your opponent may win in front of you. Just care about your speed and score.
- Your AI is always the first to draw & discard a tile.
- The score of a single round is given by:
    - `[round score] = 3 * [score of the winning hand] + [the number of tiles left in the wall]`.
    - For example, suppose your AI wins a game by completing the hand `11112345678999c` (*Full Flush, 1 Root*, 8 *Faan*), and there are 14 tiles left in the wall, then The round score will be given by `38 = 3 * 8 + 14`.
    - Definitely, you will get a `0` for **Game Draw**. And your opponent will never get any score because it never claims to win.
- Your AI will be evaluated on JOJ. For each "testcase", your AI will fight for **8 rounds**. The seed of each round is fixed for everyone. A "minimum score" is set up for each "testcase". If **the sum of 8 round scores** is no less than the "minimum score", you will "pass the testcase".

Files

Here is a brief introduction to what these files in `Lab7_Stater_Files.zip` do for the game. For details, please **refer to the file content**. There are many comments written for explanation.

- `main.cpp`
    - Contain the main function, generate two `Brain`s and the `Game`, and take the game result from the `Game`.
- `game.cpp` & `game.h`
    - Define `Result` type.
    - Declare and implement `Game` class, which controls the procedure of the game.
    - Overload I/O stream operators `<<` and `>>` for `Suit` type and `Tile` type.
- `mahjong.cpp` & `mahjong.h`
    - Define `Suit` type and `Tile` type used in `game.h` & `game.cpp`.
    - Define `Hand` class, which is specialized for calculating the score of a hand, also used in `game.cpp`. **Due to the content conflict with lab 6, the file `mahjong.cpp`, which only contains the implementation of the class `Hand`, will not be provided until Nov.30th.**
    - Feel free to use `Suit` type, `Tile` type, and `Hand` class in your brain just as they are used in `game.cpp`.
- `brain.cpp` & `brain.h`
    - **`brain.cpp` is the only file you need to modify** in this lab, where you will implement your AI by **completing 4 functions in `MyBrain` class**.
    - Your opponent `SimpleBrain` is defined and implemented in `brain.h`. You can refer to it to write your AI `MyBrain`.
    - You don't need to understand the class inheritance and virtual functions in `brain.h`, it simply enables you to write your structures without changing `brain.h`.

Task

Given `Lab7_Stater_Files.zip` on canvas, write an AI in `MyBrain` class. Please notice that since Mahjong is an imperfect information game, your AI will not have direct access to the `Game` class declared in the main

function. But it will have several interfaces between `MyBrain` and `Game`. The interfaces here are the functions you are going to implement in `MyBrain`. They are:

- `void getDraw(Tile tileDrawn)`
  - This method will be called in the Game when you draw a tile from the wall. The parameter `tileDrawn` is the tile you draw. At the beginning of a round, this method will be called 13 times to draw initial tiles.
- `Tile requestDiscard()`
  - This method will be called in the Game when you should discard a tile. You should return a tile you want to discard from your hand.
- `void getOppDiscard(Tile tileDiscarded)`
  - This method will be called in the Game when the other player discards a tile successfully. The parameter `tileDiscarded` is the tile discarded by the other player.
- `bool requestWin(Tile winningTile, bool isSelfDraw)`
  - This method will be called in the Game when you can claim for a winning. The parameter `winningTile` is the tile that can complete your hand to be a winning hand, and the parameter `isSelfDraw` is whether the type of this winning is by Self Draw (otherwise is by Discard). You should return `true` or `false` for whether agree to claim for this winning or not.

Besides the functions, you can define your structures/functions/classes/macros in `brain.cpp`, and it's better not to modify other related files because you should only submit `brain.cpp` to the JOJ platform later. The other files will be provided by us.

We have a memory & time restriction (2Mb/4000ms for each testcase), please make sure your AI won't consume too many resources.

Tips

- **Quick Start** - What should I do to the class `MyBrain`?
  - This class is empty of member variables. To enable you AI to make thoughtful decision. Definitely You need to create some member variables for it. For example, you can use `array`/`multiset`/`vector` or other containers/datatypes to store important information like the tiles in your hand, you and your opponent's river, the tile left in the wall, ... In this sense, you can simply write your `getDraw()` and `getOppDiscard()` as the storage of data. And in the could-be most complicated function `requestDiscard()`, your AI can access the class members to make wise decisions.
- **Game Trees** - How to write a traditional AI?
  - Each node illustrates one state of the game and has its utility.
  - Higher the utility, the better your result.
  - According to the current state, you choose the action that gives you the best achievable utility.
  - Obviously, time is not enough for you to traverse all possibilities in one game, Especially for Mahjong where a single turn (Draw & Discard) has at most `27*14` probabilities.
  - To deal with this time limitation, you can limit the depth of your Game Tree. Then for the leaf nodes of the tree, you need to design an **evaluation function** to calculate their utilities.
  - **If you want to learn more about Game Trees, you can check the lecture notes from CS188**, one of the most famous courses in UC Berkeley.

- **Distance** - How to evaluate an arbitrary Mahjong Hand?
    - *Distance* (向聴) is a term from Japanese Riichi Mahjong. It is used to evaluate an uncompleted hand. It is defined as the *distance* (how many tiles you need) from your hand to a **Ready Hand** (namely, a 13-tile hand that is about to win with an additional tile). The concept of *distance* may be useful for you to build your AI.
    - Before introducing how to calculate the *distance*, let's define a new term: **Partner**. A **partner**, composed of 2 tiles, is also called an "incomplete meld" (meld is either **Triplet** or **Sequence**). i.e. One tile missing for a complete meld. For example, a pair can be a **partner** because, given an additional identical tile, you will form a **Triplet**. Also, tiles like `2b4b`, `1c2c`, and `5d6d` are **partner**s too, because they can form a **Sequence** given another specific tile.
    - Let's denote, the number of melds, partners, and pairs of a hand as `nMeld`, `nPartner`, and `nPair`. You can find these numbers by decomposing your hand, and you may find many ways to decomposing your hand to count these numbers. For example, the hand `122367c1478b1389d` can be interpreted as having "1 meld (`123c`), 4 Partner (`67c`, `78b`, `13d`, `89d`), and 0 pair", or "0 meld, 5 Partner (`13c`, `67c`, `78b`, `13d`, `89d`), and 1 pair (`22c`)". For any not-winning hand, you will find some tiles that can not be counted to any of the melds, partners, or pairs. They are called **Solitary** and they contribute nothing in narrowing the *distance* of your hand.
    - Using these numbers, you can easy calculate the *distance* by the formulas below. Since there could be multiple ways to decomposing a hand, you should take the smallest *distance* as the *distance* of your hand.
    - **Formula of the Distance to Ordinary Winning Hand**
        - `distance = 8 - 2 * nMeld - nPartner - nPair`.
        - Restriction: `nPair <= 1` and `nMeld + nPartner <= 4`, otherwise they will be more melds/pairs than needed.
        - e.g. `122367c1478b1389d` has the *distance* 3.
    - **Formula of the Distance to Seven Pairs Hand**
        - `distance = 6 - nPair`.
        - Restriction: `nMeld == 0` and `nPartner == 0`, since in sever pairs we only care about pairs.
    - The *distance* of a **Ready Hand** is 0, and the *distance* of a **Winning Hand** is -1.

Grading & Submission

- This lab is worth 160 points.
- The JOJ has a scoreboard for each of you. If your AI can beat the most straightforward AI (only discard the largest tile and agree to win automatically), you can get **60 points**.
- Beat other competitors to get the other **100 points**! The official score will be given after we system-test all the submissions with more tesecases after the due date.
- After the lab, remember to compress your file `brain.cpp` into a single zip file in the following name format: lab7-[Your Last Name]-[Your First Name]-[Your SJTU ID].zip, and submit it to corresponding homework page on canvas.

Reference

- VG101 FA2021 LAB7 - TankWar, Lu Haorong
- Arcade video game. (2022, November 11). In Wikipedia. https://en.wikipedia.org/wiki/Arcade_video_game

Appendix

The pseudo-codes highlighted in yellow are where your functions are called.

